Coffee Campus App — Modelado UML y Validación Crítica con IA

Reporte Oficial de Práctica Arquitectónica y Auditoría de IA

Curso: Modelado UML y Validación Crítica con IA
Herramienta IA: Gemini (Google DeepMind)
Sistema: Coffee Campus App (Móvil)
Entregable: Fases 1, 2 y 3 (Documento Oficial)

Tabla de Contenidos:

- 1. Caso de Estudio: Coffee Campus App (Enunciado Oficial y Supuestos)
- 2. Fase 1 — Modelado Humano en Equipo (Sin IA) (Casos de Uso, Clases, Secuencia, Colaboración)
- 3. Fase 2 — Co-diseño con IA (Prompt Inicial, Iteración 1, Prompt de Corrección, Iteración 2)
- 4. Fase 3 — Entregables por Equipo (Evidencia de Renderizado, Matriz Comparativa, Conclusiones)

1. Caso de Estudio: Coffee Campus App

1.1 Enunciado Oficial (Literal del PDF)

> *"La cafetería central de la universidad requiere una aplicación móvil para reducir las filas en horas pico. El sistema debe permitir a los alumnos y profesores realizar pedidos con anticipación, pagar en línea y recoger sus productos en barra.*
>
> ***Requerimientos Funcionales:***
> ***1. Consulta de Menú:*** *Cualquier estudiante o profesor puede consultar la lista de productos disponibles y sus precios sin iniciar sesión.*
> ***2. Creación de Pedidos:*** *El cliente autenticado selecciona uno o varios productos, especifica cantidades y genera la orden.*
> ***3. Descuentos:*** *Si el cliente selecciona la opción "Descuento Estudiantil", el sistema valida la matrícula y aplica un 10% de rebaja al total antes de pagar.*
> ***4. Procesamiento de Pago:*** *Para confirmar la orden, el sistema se conecta con una Pasarela de Pago externa. Si el pago es rechazado, se notifica al cliente y el pedido queda en estado "Pendiente".*
> ***5. Preparación y Entrega:*** *Una vez confirmado el pago, el pedido pasa a la pantalla del Barista para su preparación. Al finalizar, el Barista cambia el estado a "Listo para entrega" y se notifica al cliente."*

Supuestos de Diseño del Equipo:

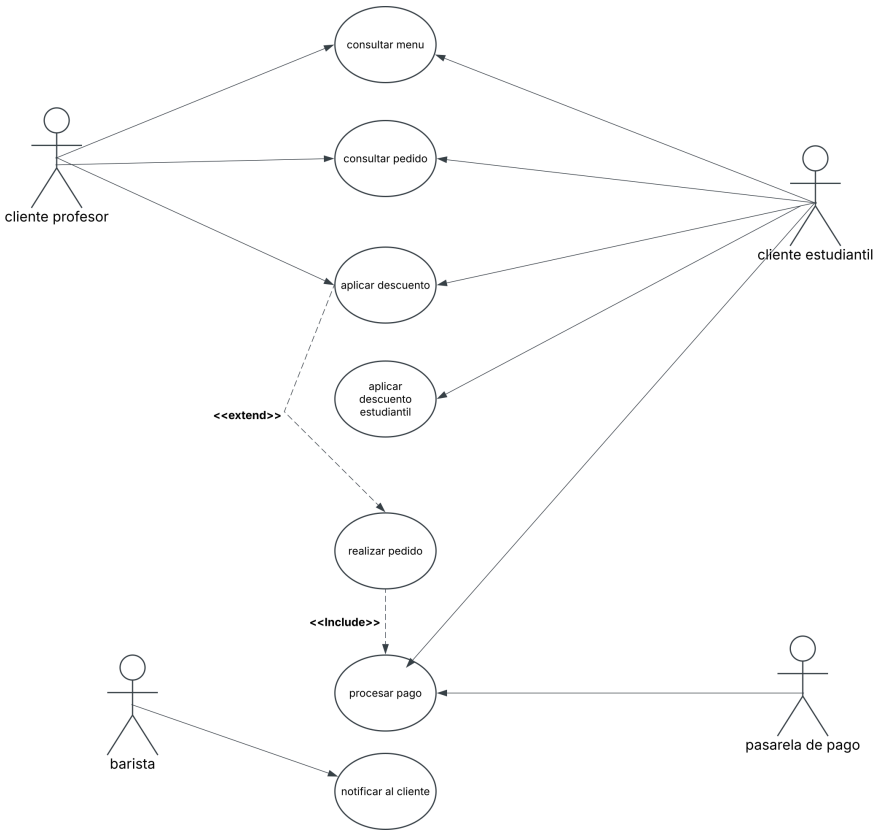
- **Validación de Matrícula (RF3):** Se asume la intervención de un servicio o actor externo (`Sistema de Matrícula`) para consultar la vigencia académica del alumno.
- **Opcionalidad del Descuento:** El beneficio de rebaja es condicional y voluntario; por ende, en casos de uso extiende (`«extend»`) la creación del pedido.
- **Acceso Abierto (RF1 vs. RF2):** La consulta de catálogo no exige credenciales; la autenticación es una inclusión (`«include»`) requerida únicamente al crear la orden.
- **Estados del Pedido (RF4 y RF5):** Si la pasarela aprueba la transacción, el pedido transiciona a `Pagado` ; si se rechaza, permanece en `Pendiente` .

2. Fase 1 — Modelado Humano en Equipo (Sin IA)

Esta sección recopila los diagramas modelados originalmente por el equipo en LucidChart, conservados en `UMLs/`, reflejando fielmente sus decisiones técnicas iniciales.

2.1 Diagrama de Casos de Uso (Diseño Original del Equipo)

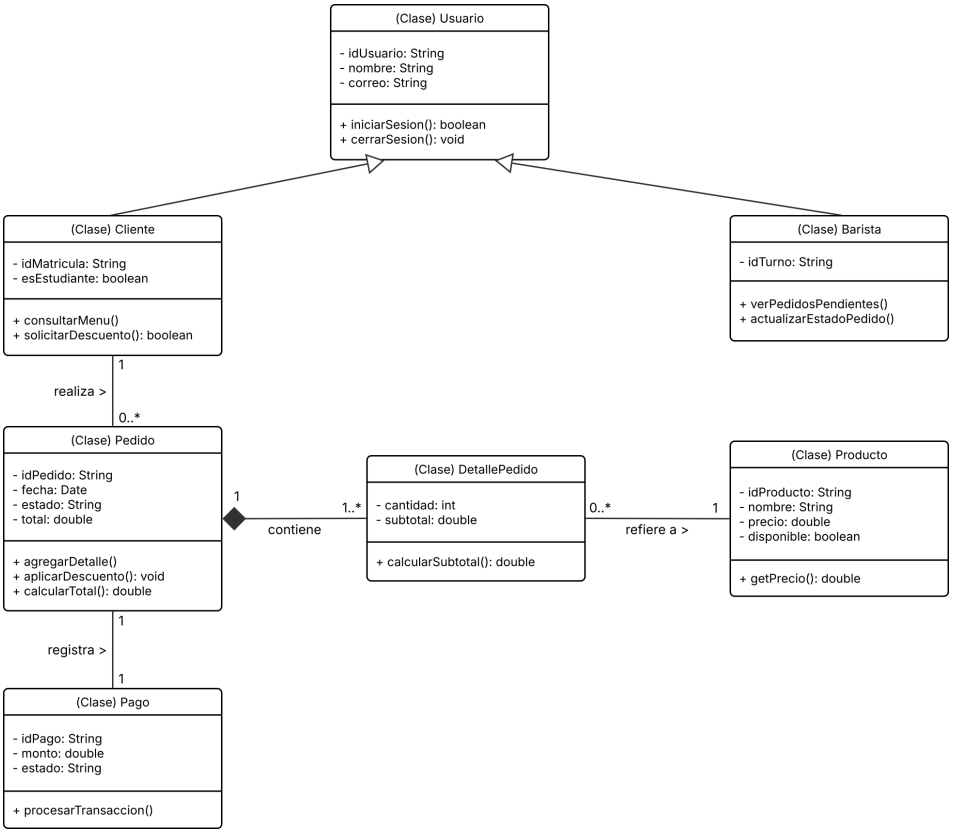
Fuente: UMLs/Casos de Uso.svg | Renderizado en alta resolución



Observaciones del diseño humano: El equipo bifurcó al cliente en dos actores independientes (`cliente profesor` y `cliente estudiantil`), omitió al actor externo `Sistema de Matrícula` , dejó huérfano el caso `aplicar descuento estudiantil` , y vinculó al barista solo con la notificación.

2.2 Diagrama de Clases (Diseño Original del Equipo)

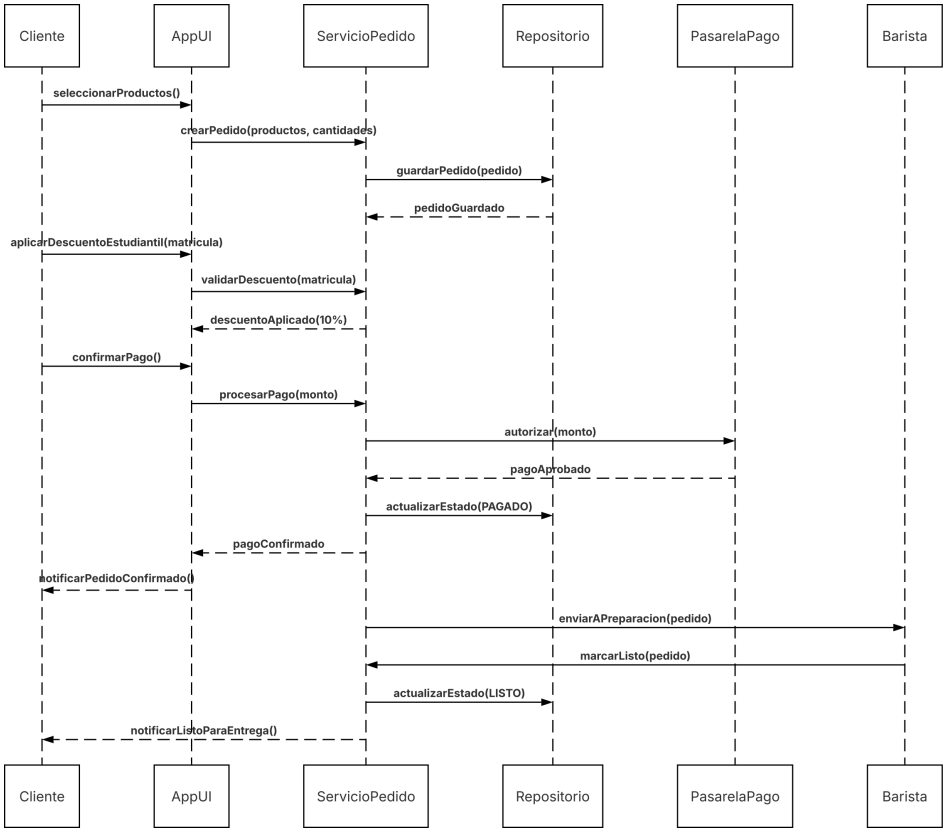
Fuente: UMLs/Clases.svg | Renderizado en alta resolución



Observaciones del diseño humano: Abstracción limpia de las 7 entidades con visibilidad estricta (+ / -), multiplicidades completas y relaciones canónicas de herencia, composición (`Pedido *-- DetallePedido`) y asociación directa.

2.3 Diagrama de Secuencia (Diseño Original del Equipo)

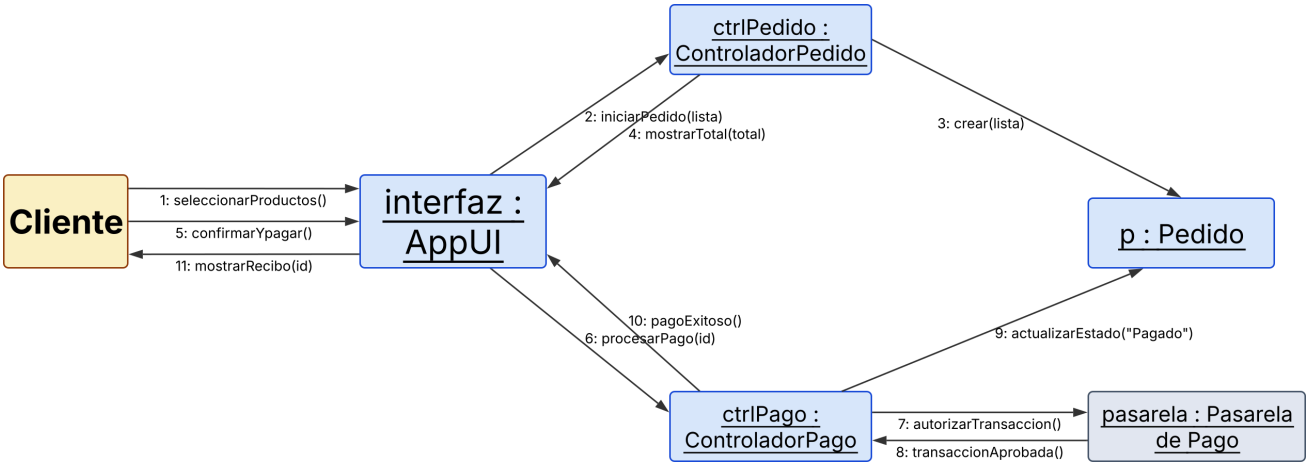
Fuente: UMLs/Secuencia.svg | Renderizado en alta resolución



Observaciones del diseño humano: Cubre el flujo de extremo a extremo (incluyendo preparación y entrega por el Barista). Empleó arquitectura en capas (`ServicioPedido` y `Repositorio`), prescindiendo de la sintaxis de instancia UML.

2.4 Diagrama de Colaboración (Diseño Original del Equipo)

Fuente: UMLs/Colaborativo.svg | Renderizado en alta resolución



Observaciones del diseño humano: Numeración cronológica clara (1 al 11) en disposición horizontal. Presentó una ruptura arquitectónica al cambiar los participantes de capas a controladores (`ctrlPedido` , `ctrlPago`) y omitir al Barista.

3. Fase 2 — Co-diseño con IA

> **3.1 Herramienta de IA Seleccionada:** Se utilizó exclusivamente **Gemini** (Google DeepMind) conforme a las directrices de la guía académica oficial.

3.2 Prompt Inicial (Parte 1 — Sin Sesgo Semántico)

Formulado con apego estricto al texto literal del PDF, evitando pistas semánticas como "extiende el flujo" para evaluar con rigor si la IA identifica autónomamente el estereotipo correcto:

```
Eres un experto en modelado UML. A partir de los siguientes requerimientos literales, genera los 4 diagramas UML en código
Mermaid, listos para renderizar en Mermaid Live:

Caso de estudio: Coffee Campus App
La cafetería central de la universidad requiere una aplicación móvil para reducir las filas en horas pico. El sistema debe
permitir a los alumnos y profesores realizar pedidos con anticipación, pagar en línea y recoger sus productos en barra.

Requerimientos Funcionales:
1. Consulta de Menú: Cualquier estudiante o profesor puede consultar la lista de productos disponibles y sus precios sin
iniciar sesión.
2. Creación de Pedidos: El cliente autenticado selecciona uno o varios productos, especifica cantidades y genera la orden.
3. Descuentos: Si el cliente selecciona la opción "Descuento Estudiantil", el sistema valida la matrícula y aplica un 10% de
rebaja al total antes de pagar.
4. Procesamiento de Pago: Para confirmar la orden, el sistema se conecta con una Pasarela de Pago externa. Si el pago es
rechazado, se notifica al cliente y el pedido queda en estado "Pendiente".
5. Preparación y Entrega: Una vez confirmado el pago, el pedido pasa a la pantalla del Barista para su preparación. Al
finalizar, el Barista cambia el estado a "Listo para entrega" y se notifica al cliente.

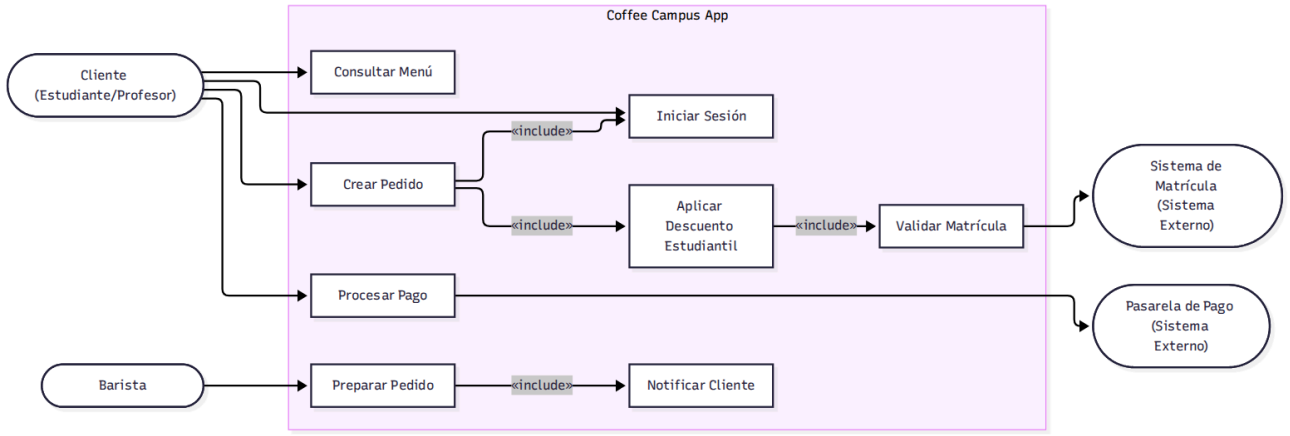
Genera:
1. Diagrama de Casos de Uso (actores principales y externos, casos de uso, «include» y «extend»).
2. Diagrama de Clases (entidades, visibilidad +/-, multiplicidades, composición/asociación/herencia).
3. Diagrama de Secuencia (escenario exitoso "Realizar Pedido y Procesar Pago", retornos y pasarela).
4. Diagrama de Colaboración (mismos objetos, numeración cronológica explícita sobre enlaces).
```

3.3 Diagramas Generados por la IA — Primera Iteración

Generados directamente por Gemini en este chat a partir del prompt inicial sin sesgo:

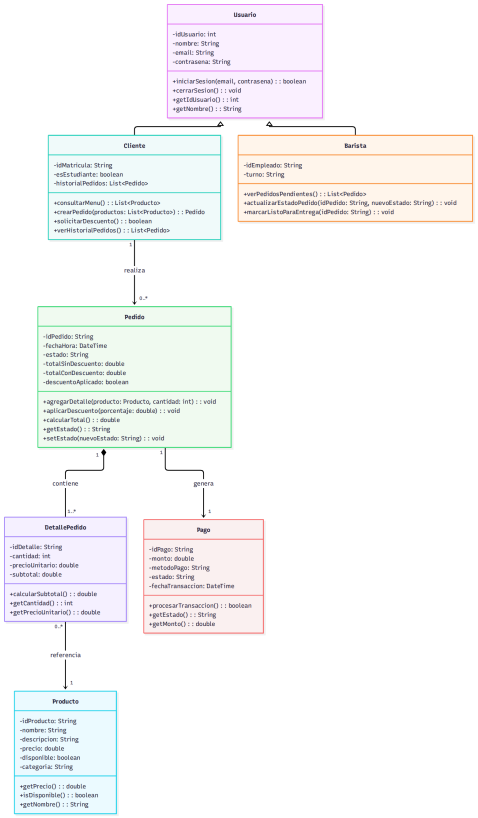
3.3.1 Casos de Uso (IA - Iteración 1)

Renderizado: img/05_ia_it1_casos_de_uso.png



3.3.2 Clases (IA - Iteración 1)

Renderizado: img/06_ia_it1_clases.png



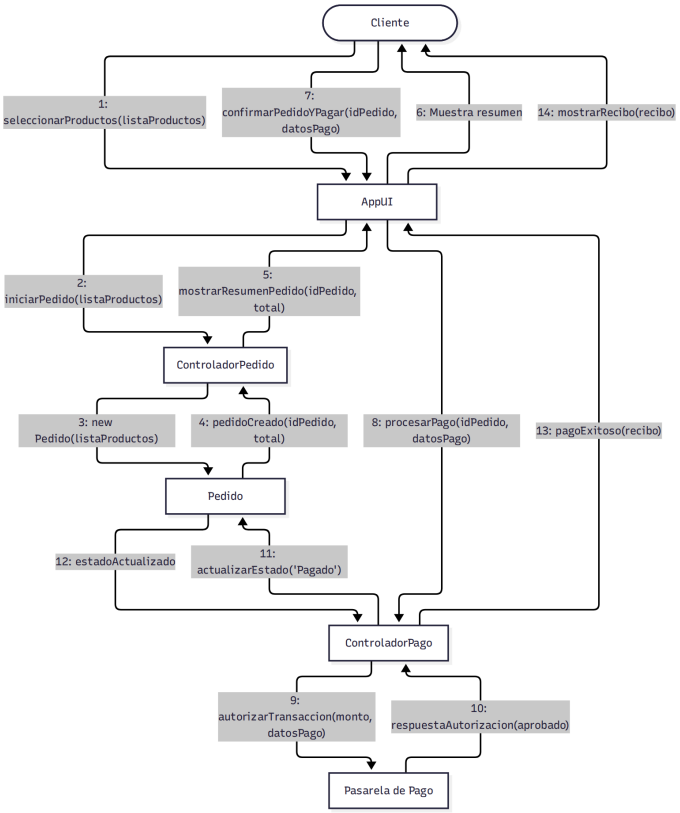
3.3.3 Secuencia (IA - Iteración 1)

Renderizado: img/07_ia_it1_secuencia.png



3.3.4 Colaboración (IA - Iteración 1)

Renderizado: img/08_ia_it1_colaboracion.png



3.4 Prompt de Corrección (Parte 3 — Feedback Iterativo del Equipo)

El equipo humano realizó **una única ronda de feedback estructurado** señalando las reglas formales de OMG UML 2.5.1 infringidas, sin dictarle el código ni regalarle la solución:

```
Revisamos tus diagramas UML y encontramos errores conceptuales y de notación que debes corregir de acuerdo al estándar OMG UML:

1. En el Diagrama de Casos de Uso:
   - Error semántico en relaciones: Marcaste la relación entre "Crear Pedido" y "Aplicar Descuento Estudiantil" como «include». Según la especificación UML, «include» impone obligatoriedad condicional incondicional, mientras que el descuento es opcional y por demanda. Debe modelarse como «extend», con la dirección de flecha correcta (desde el caso de uso que extiende hacia el caso base).
   - Error de alcance de actores: Conectaste al Cliente directamente con "Iniciar Sesión". El inicio de sesión no es un objetivo de negocio independiente en este caso, sino una condición obligatoria incluida en "Crear Pedido".
   - Modela la relación de generalización entre Profesor, Estudiante y Cliente, y asegura que el rechazo de pago quede reflejado.

2. En el Diagrama de Clases:
   - Sobre-especificación y duplicación: Agregaste atributos no contemplados en el dominio (como contrasena, historialPedidos, categoria, descripcion) y saturaste las entidades con getters/setters triviales. Limítate a los datos del problema.
   - En UML, no debes duplicar una relación de asociación como un atributo de colección dentro de la clase (como historialPedidos en Cliente).
   - Revisa la multiplicidad Pedido-Pago: un pedido rechazado o pendiente no posee un pago confirmado; ajusta la cardinalidad para reflejar 0..1.
   - Modela explícitamente la relación del Barista con el Pedido e incluye una relación de agregación en el dominio justificando su uso frente a la composición de DetallePedido.

3. En el Diagrama de Secuencia:
   - Notación de instancias: No nombraste las líneas de vida con la convención estándar UML 'nombreInstancia : NombreClase' (o ':Clase').
   - Desviación de alcance: Se te pidió exclusivamente el escenario exitoso "Realizar Pedido y Procesar Pago", pero incluiste un bloque alt con manejo de errores que desvirtúa el flujo principal.
   - Creación de objetos: Modela la instanciación de 'p : Pedido' usando la sintaxis formal de creación de participantes ('create participant'). Incluye activación/desactivación de líneas de vida.

4. En el Diagrama de Colaboración:
   - Inconsistencia de paso de mensajes: Enumeraste respuestas de retorno sincrónicas como pasos secuenciales nuevos (14 pasos), lo cual confunde invocaciones con confirmaciones. Agrupa o representa los mensajes sobre los mismos enlaces.
   - Usa orientación horizontal (flowchart LR) para evitar cruce de líneas y mantén consistencia absoluta con los mismos participantes y orden del diagrama de secuencia.

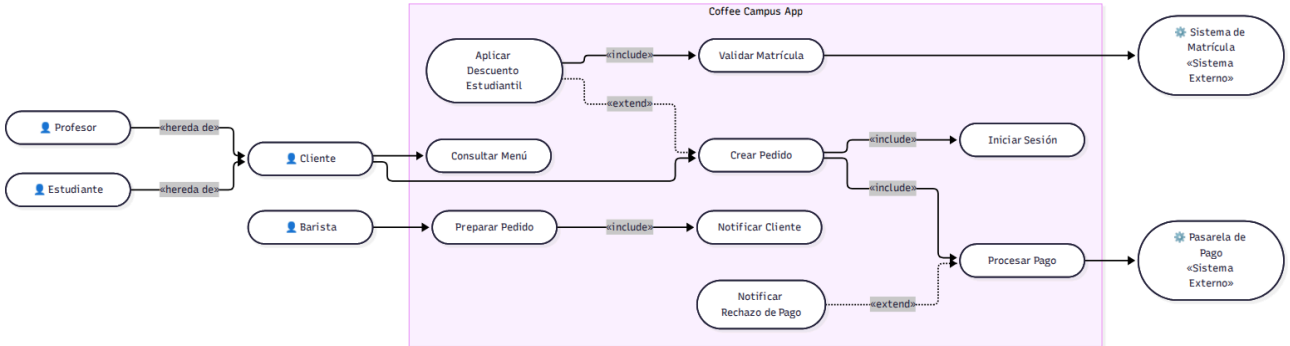
Regenera los 4 diagramas en código Mermaid corrigiendo estos puntos sin agregar información fuera del alcance.
```

3.5 Diagramas Corregidos por la IA — Segunda Iteración

Diagramas corregidos generados por Gemini tras procesar el feedback de ingeniería de prompts:

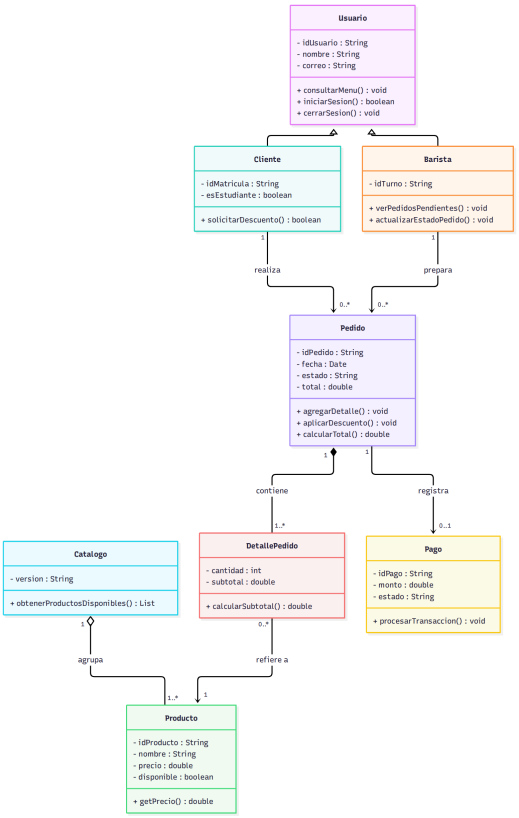
3.5.1 Casos de Uso (Corregido — Iteración 2)

Renderizado: img/09_ia_it2_casos_de_uso.png



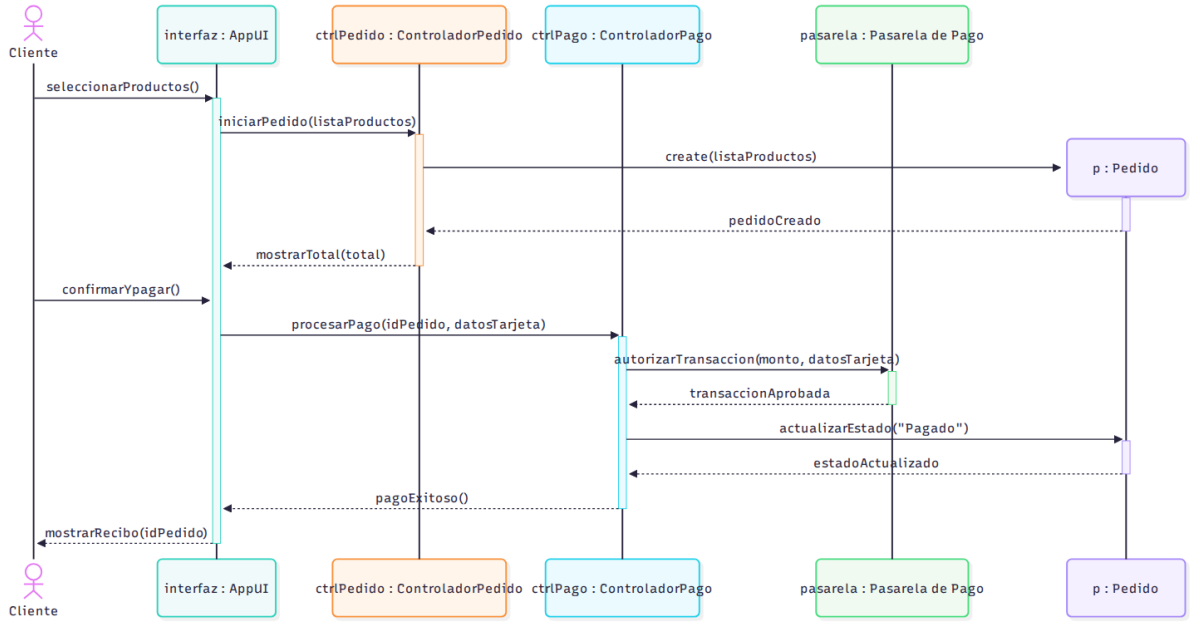
3.5.2 Clases (Corregido — Iteración 2)

Renderizado: img/10_ia_it2_clases.png



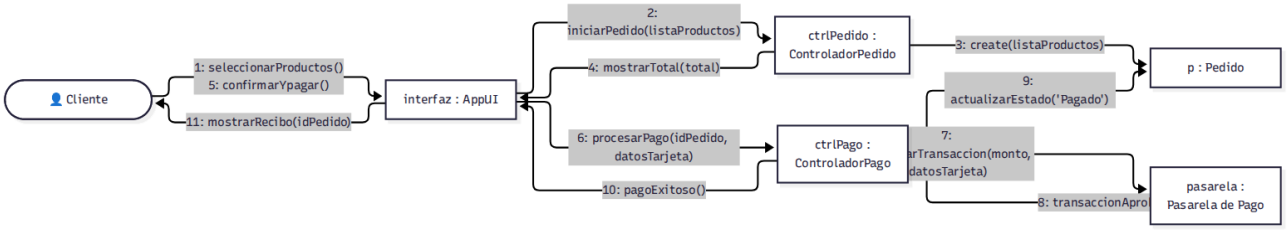
3.5.3 Secuencia (Corregido — Iteración 2)

Renderizado: img/11_ia_it2_secuencia.png



3.5.4 Colaboración (Corregido — Iteración 2)

Renderizado: img/12_ia_it2_colaboracion.png



4. Fase 3 — Entregables por Equipo

4.1 Evidencia de Renderizado en Mermaid Live y Exportación

Todos los diagramas del proyecto se validaron y compilaron en archivos de alta resolución mediante `@mermaid-js/mermaid-cli` (`mmdc`) y `rsvg-convert` :

Diagrama	Diseño Original (Equipo)	Iteración 1 (Salida IA)	Iteración 2 (IA Corregida)
Casos de Uso	`img/01_humano_casos_de_uso.png`	`img/05_ia_it1_casos_de_uso.png`	`img/09_ia_it2_casos_de_uso.png`
Clases	`img/02_humano_clases.png`	`img/06_ia_it1_clases.png`	`img/10_ia_it2_clases.png`
Secuencia	`img/03_humano_secuencia.png`	`img/07_ia_it1_secuencia.png`	`img/11_ia_it2_secuencia.png`
Colaboración	`img/04_humano_colaboracion.png`	`img/08_ia_it1_colaboracion.png`	`img/12_ia_it2_colaboracion.png`

4.2 Matriz de Análisis Comparativo

Estructura oficial del PDF evaluando la solución original del equipo frente a la generación inicial de la IA y su diagnóstico:

Aspecto Evaluado	Diagrama Original (Equipo)	Diagrama Generado (IA)	Hallazgo / Error de la IA
Casos de Uso *(¿Confundió include con extend?)*	El equipo modeló `«extend»` hacia `realizar pedido` , pero separó `cliente profesor` y `cliente estudiantil` como actores independientes y omitió el actor externo `Sistema de Matrícula` . Dejó `aplicar descuento estudiantil` como caso aislado.	**Sí, confundió gravemente los estereotipos:** conectó `Crear Pedido` con `Aplicar Descuento` mediante `«include»` , tornando el descuento obligatorio. Además, creó una asociación directa innecesaria entre `Cliente` e `Iniciar Sesión` .	La IA trató como relación obligatoria (`«include»`) una extensión de negocio puramente opcional (`«extend»`). Tuvo la ventaja de unificar al cliente y detectar el sistema de matrícula externo, pero falló en la semántica relacional.
Clases *(¿Omitió visibilidad o inventó atributos?)*	Modeló las 7 clases nucleares con visibilidad estricta (+ / -), multiplicidades completas y tipos estándar (`Date`).	Respetó la visibilidad básica, pero **inventó atributos no requeridos** (`contrasena` , `historialPedidos` , `categoria` , `descripcion`), usó tipos de frameworks (`DateTime`), saturó con getters/setters y duplicó asociaciones como atributos.	**Sobre-especificación (Feature Creep):** La IA introdujo ruido con patrones de código comercial no solicitados en vez de ceñirse a las entidades conceptuales del enunciado.
Secuencia *(¿Olvidó mensajes de retorno o la pasarela?)*	Modeló extremo a extremo (incluyendo Barista y `Repositorio`), con retornos y pasarela, pero sin notación de instancia formal (`instancia : Clase`).	**No olvidó los retornos ni la pasarela** (ambos estuvieron presentes). Sus errores reales fueron: omitió activaciones de línea de vida, omitió la notación canónica `instancia : Clase` y añadió un bloque `alt` no pedido.	**Desviación de alcance y notación:** La IA mezcló el escenario de falla en una solicitud de escenario exitoso y prescindió de la sintaxis formal de instancias UML.
Colaboración *(¿Logró numerar el orden de mensajes sobre enlaces?)*	Numeró enlaces del 1 al 11 en orientación `LR` , pero con una **inconsistencia arquitectónica grave**: utilizó participantes distintos a su propio diagrama de secuencia (`ControladorPedido` vs `ServicioPedido`).	**Numeró los mensajes, pero de forma errónea:** asignó números secuenciales independientes a las respuestas y retornos sincrónicos (14 pasos) y utilizó layout vertical `TD` de difícil lectura.	**Confusión de mensajes vs retornos:** La IA no distinguió entre llamadas a métodos y respuestas implícitas, saturando el grafo de flechas redundantes.

4.3 Evaluación Cruzada de Errores de Modelado

Deficiencias del equipo humano: Duplicación de actores principales por rol laboral en vez de generalización; omisión del actor externo de matrícula; e incoherencia de patrones arquitectónicos entre secuencia (Service-Repository) y colaboración (Controller-Entity).

Deficiencias de la IA: Incapacidad inicial para distinguir obligatoriedad condicional (`«include»` vs. `«extend»`); sobre-ingeniería orientada a código de producción; modelado redundante de confirmaciones en grafos de colaboración; e inclusión espontánea de flujos alternativos no pedidos.

4.4 Conclusión: Semántica UML vs. Sintaxis en IA Generativa

La evaluación comparativa demostró que los modelos de IA generativa poseen una competencia sintáctica sobresaliente en la producción de código declarativo para diagramas (como Mermaid), pero presentan dificultades recurrentes para discernir la semántica rigurosa del estándar UML. Mientras que la estructura y renderizado fueron correctos en un primer intento, la IA confundió dependencias obligatorias (`«include»`) con extensiones opcionales (`«extend»`), sobrecargó entidades con atributos no requeridos e infló diagramas de colaboración al tratar retornos sincrónicos como llamadas independientes. Estas observaciones evidencian que las herramientas de IA son eficaces para acelerar la escritura visual y proponer estructuras base, pero demandan la intervención crítica del modelador humano para validar la fidelidad al dominio, las restricciones arquitectónicas y la consistencia formal del sistema.